



# Adversarial imitation learning with deep attention network for swarm systems

Yapei Wu<sup>1</sup> · Tao Wang<sup>1</sup> · Tong Liu<sup>1</sup> · Zhicheng Zheng<sup>1</sup> · Demin Xu<sup>1</sup> · Xingguang Peng<sup>1</sup>

Received: 23 June 2024 / Accepted: 11 October 2024 / Published online: 12 November 2024  
© The Author(s) 2024

## Abstract

Swarm systems consist of a large number of interacting individuals, which exhibit complex behavior despite having simple interaction rules. However, crafting individual motion policies that can manifest desired collective behaviors poses a significant challenge due to the intricate relationship between individual policies and swarm dynamics. This paper addresses this issue by proposing an imitation learning method, which derives individual policies from collective behavior data. The approach leverages an adversarial imitation learning framework, with a deep attention network serving as the individual policy network. Our method successfully imitates three distinct collective behaviors. Utilizing the ease of analysis provided by the deep attention network, we have verified that the individual policies underlying a certain collective behavior are not unique. Additionally, we have analyzed the different individual policies discovered. Lastly, we validate the applicability of the proposed method in designing policies for swarm robots through practical implementation on swarm robots.

**Keywords** Swarm system · Adversarial imitation learning · Deep attention network · Swarm robots

## Introduction

Swarm systems are a prevalent phenomenon in both natural and human societies. Examples include animal collective movement [1], bacterial collective dynamics [2], human crowds [3], and even market dynamics [4]. These systems exhibit coordinated and orderly collective behavioral patterns, such as flocking and vortex patterns. Research indicates that these behaviors emerge as a result of interactions among individuals within the swarm, where simple local interactions among individuals lead to complex collective behavior, a phenomenon known as emergence. Swarm systems have been successfully deployed across various domains, including military [5], agriculture [6], environmental monitoring [7], and construction [8].

Designing individual policies to achieve desired collective behaviors is a challenging task due to the nontrivial relationship between individual policies and collective behavior. Swarm systems typically consist of numerous individu-

als interacting with simple policies. Even though these policies are straightforward, interactions among individuals can lead to complex collective behaviors [9]. Consequently, developing swarm controllers often requires iterative approaches, such as reinforcement learning-based methods [10–12] and evolutionary computation-based methods [13, 14]. These methods rely on the designer's intuition and experience to formulate objective functions (or reward functions). Subsequently, through continuous experimentation and adjustment, individual policies that meet the objectives are identified. However, designing suitable objective functions is a challenging task.

In recent years, there has been development in the application of imitation learning methods to swarm systems. These methods aim to learn individual local policy models by imitating collective behaviors. Unlike traditional reinforcement learning methods, imitation learning bypasses the need for manually designing reward functions, thereby alleviating the limitations of expert-designed reward functions.

Existing imitation learning methods for swarm systems include behavioral cloning (BC) [15] and inverse reinforcement learning (IRL) [16]. BC methods learn demonstration behavior in a supervised learning manner, minimizing the discrepancy between demonstration actions and actions generated by the policy. Works such as [17–19] have used

✉ Xingguang Peng  
pxg@nwpu.edu.cn

<sup>1</sup> School of Marine Science and Technology, Northwestern Polytechnical University, 127 Youyixi Road, Xi'an 710072, Shaanxi, China

networks like GNN and LSTM to learn policies for swarm systems. However, BC methods suffer from compounding errors [20]. A large amount of demonstration data is required to achieve the desired performance. IRL method assumes that the policy implicit in the demonstration data is the optimal policy for a certain reward function. The policy in the demonstration data is imitated by first recovering the reward function and then training the corresponding policy. Studies like [21–23] have applied IRL to imitate the behavior of pigeon flocks and the Vicsek model. Nevertheless, the computational complexity of IRL is high, and certain prior knowledge is required in designing the reward function.

The adversarial imitation learning (AIL) methods are a class of imitation learning methods based on generative adversarial networks. AIL simultaneously learns the reward function and the policy, offering lower computational complexity compared to IRL. Additionally, AIL does not have the compound error problem. By using neural network as reward function, no a priori knowledge is required. In this paper, we use deep attention network [24] as an individual policy network in the AIL framework. The deep attention network comprises one pairwise interaction network and one aggregation network. The former learns the interactions between two individuals, while the latter provides weights for the influence of each neighbor on the focal individual, serving as attention mechanisms. This modularity results in a low dimension (i.e., low variable-complexity) of inputs and outputs in each network. The inputs of each network depend on the state of a single neighbor, and the outputs represent the focal individual's response to that neighbor. Given that the input and output dimensions are invariant to swarm size, the deep attention network is adaptable to swarm systems of varying scales. Additionally, the low dimension of inputs and outputs facilitates the analysis of the relationships between inputs and outputs. This allows us to gain insights into the hidden policies underlying the demonstration collective behaviors while imitating them.

This paper contributes in the following three aspects:

1. An adversarial imitation learning method using a deep attention network as the policy is developed and applied to a collective behavior imitation task. The imitation of three typical collective behaviors is achieved.
2. We verified that the individual policies underlying a certain collective behavior are not unique. By taking advantage of the model's modularity, we graphically analyzed several imitated models. We found that the strategies hidden in the demonstration data could be diverse.
3. By conducting the imitated strategies on a physical robot swarm platform, the feasibility of deriving control policies for swarm robots from data has been confirmed.

## Preliminaries and related work

### Preliminaries

Imitation Learning, also known as Learning from Demonstration (LfD), does not rely on reward signals  $r$  from the environment. Instead, it learns a policy by imitating the expert policy  $\pi^E$  based on expert demonstrations data  $\xi^E$ .

Adversarial Imitation Learning (AIL) is a kind of promising imitation learning method, it employs an adversarial training framework, enabling simultaneous learning of reward functions and policy optimization. Generative Adversarial Imitation Learning (GAIL) [25] first introduced Generative Adversarial Networks (GANs) to tackle imitation learning tasks. The GAN consists of a generator and a discriminator. The task of the generator is to produce samples to deceive the discriminator, while the discriminator aims to distinguish between samples generated by the generator and those provided by experts. In GAIL, the policy  $\pi_\theta$  acts as the generator, imitating expert behaviors and interacting with the environment to generate data, while the discriminator  $D_\beta(s, a): \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$  is used to differentiate between expert data samples  $(s^E, a^E) \sim \rho^E$  and policy-generated samples  $(s, a) \sim \rho^\pi$ .

The training process of GAIL involves alternating iterations between the discriminator and the generator. The optimization objective of the discriminator is defined based on the binary classification cross-entropy loss  $\mathcal{L}_\beta$ , given by:

$$\min_{\beta} \{ \mathcal{L}_\beta \} = \min_{\beta} \{ \mathbb{E}_{\pi} [-\log(D_\beta(s, a))] + \mathbb{E}_{\pi^E} [-\log(1 - D_\beta(s, a))] \} \quad (1)$$

Meanwhile, the objective of the generator is to maximize the expected cumulative reward:  $\max_{\pi} \{ \mathbb{E}_{s, a \sim \rho_0, \pi} [P \sum_{t \geq 0} \gamma^t r(s_t, a_t)] \}$ . Here, the reward signal is a function of the discriminator's output, i.e.,  $r(s, a) = -\log D_\beta(s, a)$ . Therefore, the overall objective function of GAIL can be expressed as:

$$\min_{\pi} \max_D \mathbb{E}_{\pi} [\log(D(s, a))] + \mathbb{E}_{\pi^E} [\log(1 - D(s, a))] - \lambda H(\pi) \quad (2)$$

Here,  $\lambda$  is a hyperparameter, and  $H(\pi)$  represents the entropy of the policy  $\pi$ .

### Related work

In recent years, various imitation learning methods for collective behavior have been developed, aiming to design policies for individuals in a swarm or to analyze biological collec-

tive behavior. These methods are mainly divided into two categories: one is the behavior cloning method based on supervised learning, and the other is the inverse reinforcement learning method, which alternately learns individual reward functions and individual policies.

Behavior cloning methods aim to learn an individual policy from demonstration data by minimizing the discrepancy between actions taken by the policy and those in the demonstration data at the corresponding states. While behavioral cloning methods are straightforward to implement, their performance heavily relies on the quantity and quality of training data. This dependence stems from the limited exploration capability of the behavioral cloning algorithm, resulting in poor generalization. Compound errors [20] occur when confronted with states that do not exist in the demonstration data.

As one approach rooted in behavioral cloning, Guo et al. [18] utilize graph neural networks for representation learning, specifically, to learn the states on which individual decisions depend. These states may include information about the individual's own position, velocity, etc., and those of neighboring individuals. By minimizing the mean squared error between demonstration actions and imitated policy actions, they learn a flocking control mechanism. Zhou et al. [17] proposed SwarmNet, which utilizes one-dimensional convolution and graph convolution to extract features from the positions and velocities of neighboring individuals. Subsequently, linear layers are used for decision-making to imitate individual policy in collective behavior. Ekaterina et al. [26] employed an aggregated graph neural network [27] to learn individual control strategies that only require local communication, by imitating a centralized control policy. The effectiveness of the algorithm was validated on a collective flocking behavior.

However, graph neural networks suffer from high computational complexity, which exponentially increases with the number of individuals, particularly in large-scale swarm systems, leading to low efficiency in training and inference. Moreover, graph neural networks are sensitive to changes in graph structure and may lack robustness to dynamic graph structures. Consequently, Fabian et al. [28] implemented a vision-based collective behavior imitation learning method, circumventing the use of graph neural networks. Fabian et al. utilized visual images from quadcopter drones as input and employed convolutional neural networks for representation learning. By matching the actions of the Reynolds flocking algorithm, they imitated a flocking control mechanism that solely relies on visual images without the need for inter-individual communication of positional information. Both graph neural networks and convolutional neural networks have complex structures that make quantitative analysis difficult. To gain insights into individual policies, Heras et al.

[24] adopted a modular deep attention network architecture to learn from zebrafish schooling data. By employing this deep attention network, they examined the relationship between the inputs and outputs of each module to gain insights into zebrafish collective behavior.

IRL for swarm system involves firstly learning the underlying reward function of individuals from demonstration data and then maximizing this reward to learn the individual policies. When applied to the problem of imitating the collective behavior, IRL aims to learn an individual reward function that can maximally distinguish between demonstrated and generated collective behaviors. Subsequently, this reward function is utilized to train the individual policy, thereby achieving imitation of the collective behavior.

In recent years, there have been several studies using inverse reinforcement learning for collective behavior imitation. Adrian et al. [21] firstly extended the IRL method to large-scale swarm behavior imitation. They utilized a linear combination of individual state features as the reward function and optimized its weights to learn the reward function. The optimization objective aimed to maximize the difference between demonstration data and generated data. Following this, individual strategies were trained with the objective of maximizing individual rewards. Their proposed method successfully imitated the flocking behavior of the Vicsek model [29]. Yet their method is only applicable to discrete action spaces. Pinsle et al. [23] used IRL to study the flocking behavior of a pigeon swarm. They designed a series of collective behavior statistics as features in the reward function. Based on the learned individual reward function, this method learned the flocking behavior of the pigeon swarm and inferred the leader-follower hierarchy within the pigeon swarm. The above inverse reinforcement learning based methods typically rely on empirical knowledge to design the features in the reward function. The method proposed by Yu et al. [22] uses a neural network as the reward function, eliminating this requirement. Their method is validated on the flocking behavior of the Vicsek model.

In this paper, we apply deep attention networks to adversarial imitation learning (AIL) methods. Our approach combines the exploration ability of AIL and the modularity of deep attention networks. AIL explore the policy space through interactions with the environment, reducing the impact of compounding errors. Consequently, AIL can achieve satisfactory performance even with few demonstration data. Additionally, the models learned by deep attention networks are easily analyzable. This is because each module of the network has few input and output variables. As a result, we are able to gain insights into complex swarm systems with limited demonstration data.

## Method

### The structure of the policy network

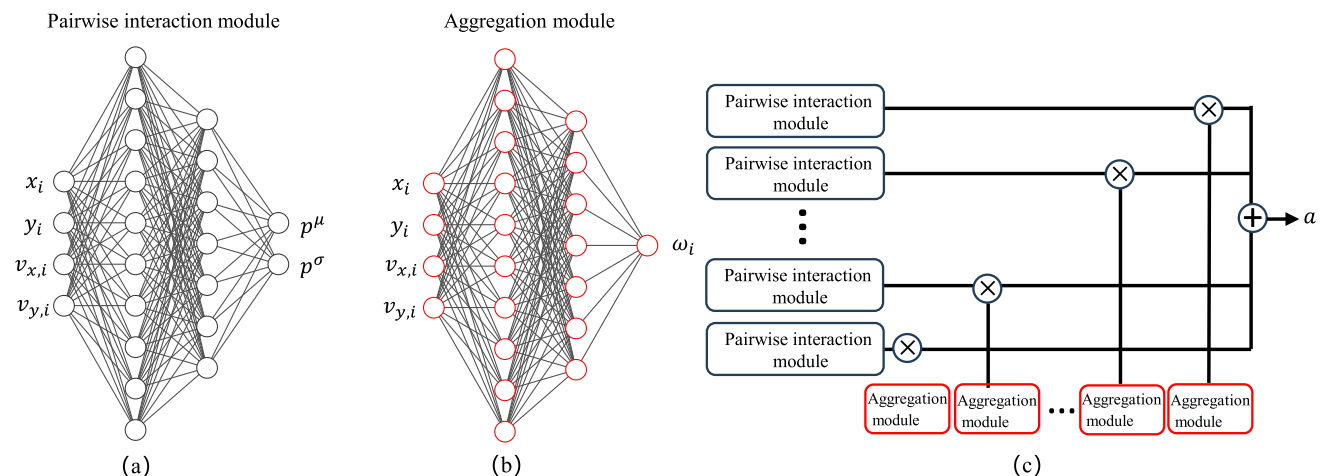
In this paper, the individual policy network is a modular neural network, referred to as a deep attention network in the relevant literature [24, 30–32]. Figure 1 illustrates the complete structure of a deep attention network, which consists of two key components: the pairwise interaction network and the aggregation network. The individual policy network in this paper is denoted as  $\pi_{\phi, \varphi}$ , where  $\phi$  represents the parameters of the pairwise interaction network, and  $\varphi$  represents the parameters of the aggregation network. The role of the pairwise interaction network is to capture and encode the interactions between individuals, facilitating the comprehension of dynamics and interactions within the swarm. The aggregation network, on the other hand, is responsible for assigning weights to these interactions so that they can be integrated to form a comprehensive understanding of the overall collective behavior.

Both the pairwise interaction network and the aggregation network have the same inputs, which are the distances and the relative velocities of neighbors with respect to the focal individual. Because the individual speed is fixed in this study, we do not use the velocity of the focal individual as input, which differs from the settings in related work [30, 31].

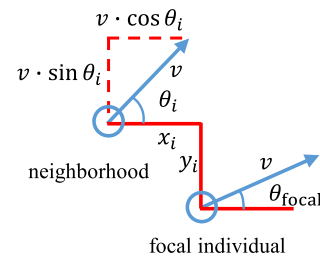
$$x_i = x_{neighbor} - x_{focal} \quad (3)$$

$$y_i = y_{neighbor} - y_{focal} \quad (4)$$

$$\begin{bmatrix} v_{x,i} \\ v_{y,i} \end{bmatrix} = \begin{bmatrix} \cos(-\theta_{focal}) & -\sin(-\theta_{focal}) \\ \sin(-\theta_{focal}) & \cos(-\theta_{focal}) \end{bmatrix} \begin{bmatrix} v \cos(\theta_i) \\ v \sin(\theta_i) \end{bmatrix} \quad (5)$$



**Fig. 1** The structure of the policy network. **a** Pairwise interaction network, with inputs being the position  $x_i$  and  $y_i$  of neighbor  $i$ , as well as the velocity components  $v_{x,i}$  and  $v_{y,i}$  along the  $x$  and  $y$  axes. The outputs are the mean  $p^\mu$  and variance  $p^\sigma$  of actions influenced by neighbor  $i$ . **b** Aggregation network, with inputs identical to those of



**Fig. 2** The definition of state for each individual

Here,  $x_{focal}$ ,  $y_{focal}$  represent the global coordinates and velocities of the focal individual, and  $x_{neighbor}$ ,  $y_{neighbor}$  represent the global coordinates of neighboring individuals. The  $\theta$  denotes the individual heading angle, calculated with respect to the  $x$ -axis as the reference direction. According to Eq. (5), the velocity components of neighboring individuals,  $v \cos(\theta_i)$ ,  $v \sin(\theta_i)$ , are transformed into the relative velocities  $v_{x,i}$  and  $v_{y,i}$  with respect to the focal individual. The definitions of each state variable are illustrated in Fig. 2.

The outputs of the pairwise interaction network and the aggregation network differ. Specifically, the pairwise interaction network learns the interaction between the focal individual and its neighbor  $i$ , characterized by the change in the focal individual's heading angle, denoted as the action  $a_i$ . The outputs of the pairwise interaction network are the mean  $p^\mu$  and the variance  $p^\sigma$  of the turning angle (see Fig. 1a). The individual's action  $a_i$  is sampled based  $p^\mu$  and  $p^\sigma$ :  $a_i \sim N(p_i^\mu, p_i^\sigma)$ . Then, the action  $a$  is normalized to the range  $[0, 1]$  using the Sigmoid activation function. The aggregation network learns the magnitude of influence that neighbor  $i$  exerts on the focal individual, as indicated by

the pairwise interaction network and outputs representing the weights  $\omega$  influenced by neighbor  $i$ . **c** The complete policy network is the sum of the element-wise multiplication of outputs from the pairwise interaction and aggregation networks, denoted as  $a$ , which represents the next steering action of the focal individual

the weight  $\omega_i$ . The output of the aggregation network is the weight  $\omega$  (see Fig. 1b). Due to the existence of the aggregation network, the magnitude of the influence of different neighbors on the focal individual varies, akin to an attention mechanism.

The complete structure of the policy network is illustrated in Fig. 1c. The policy network's ultimate output is the sum of the products of actions  $a_i$  and  $\omega_i$  obtained after passing the states of all  $N$  neighbors ( $\mathcal{I}$ ) through both the pairwise interaction network and the aggregation network modules. The weights  $\omega_i$  are normalized to ensure that the sum of all  $\omega_i$  equals 1. The final formula is:

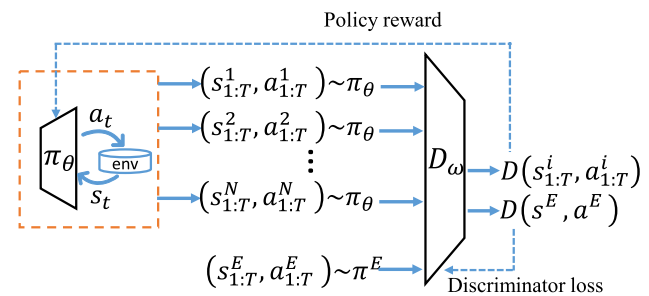
$$a = \sum_{i \in \mathcal{I}} a_i \frac{\omega_i}{\sum_{j \in \mathcal{I}} \omega_j} \quad (6)$$

This modularized network provides a powerful tool for the research of the swarm system. The design of the individual policy network in the swarm system requires a thorough understanding and modeling of the complexity of collective behavior, enabling them to make globally reasonable decisions based on local information. This modularized structure simultaneously models the mutual influence between individuals and individual's perception of the global context. Additionally, the modularized structure can flexibly adapt to swarm systems of different scales. Compared to non-modularized networks, this modularized model reduces the input dimensionality. For example, in a scenario with 20 neighbors, if a non-modularized network were used,  $20 \times 4 = 80$  variables would be required as inputs for individual policies. Studying interactions between individuals within 80 dimensions is difficult. Utilizing a deep attention network, each module relies on only 4 input variables, facilitating independent analysis of each module. The analysis of each module enables us to delve deeper into understanding collective behavior.

### Framework of adversarial imitation learning for swarm system

This paper employs an adversarial imitation learning framework to imitate self-organized collective behavior. The method aims to learn individual policies that can generate self-organized collective behavior similar to demonstration trajectories. In the setting of collective behavior imitation, due to the homogeneity of individuals, individuals adopt the same policy, resulting in  $N$  trajectories  $\{\tau_1, \tau_2, \dots, \tau_N\}$  generated by the policy  $\pi$ , all of which can be considered as trajectories generated by policy  $\pi$ .

The proposed method consists of a discriminator network  $D$  and a policy network  $\pi$ , as illustrated in Fig. 3. The discriminator is trained based on the discriminator loss, aiming to differentiate between demonstration data and



**Fig. 3** The framework of adversarial imitation learning for swarm system

policy-generated data. The policy is iteratively updated based on the received rewards, gradually matching the generated collective behavior with that of the demonstrated collective behavior. In this paper, an evolutionary strategy called OpenAI-ES [33] is employed as the optimization algorithm to update the policy network.

Algorithm 1 describes the flow of the proposed adversarial imitation learning method. The algorithm takes demonstration data and initial networks as input and ultimately outputs an individual policy network  $\pi_{\pi, \varphi}$  capable of imitating demonstrated collective behavior. Firstly, the algorithm utilizes the initial policy network to execute simulations of collective behavior, generating initial collective trajectories and storing these data in a generated data buffer (Steps 1–2). The main loop of the algorithm involves alternating updates between the discriminator network and the policy network for  $M$  iterations (Steps 3–9). Specifically, the discriminator samples generated data from the generated data buffer, computes the loss gradients of the network parameters using both generated and demonstrated data, and then updates the network parameters accordingly (Steps 4–5). Subsequently, an evolutionary strategy algorithm is employed to update the parameters of the pairwise interaction networks and the aggregation network separately. Trajectories generated by the policy during this process are stored in the generated data buffer (Steps 6–8).

#### 1. Loss function of the discriminator network

The discriminator is trained with both generated and demonstration data in a supervised manner, aiming to better distinguish between the two types (assigning higher scores to demonstration data and lower scores to generated data). To enhance the stability of adversarial training, this paper employs a discriminator loss function with a gradient penalty term:

$$\mathcal{L}_{\omega}^{GP} = \mathbb{E}_{\pi} [-\log(D_{\beta}(s, a))] + \mathbb{E}_{\pi_E} [-\log(1 - D_{\beta}(s, a))] + \lambda \mathcal{L}_{GP} \quad (7)$$

where  $D_{\beta}$  represents the discriminator,

---

**Algorithm 1:** Adversarial Imitation Learning for Swarm Systems
 

---

**Input** : Initial pairwise interaction network parameters  $\theta_0$ , aggregation network parameters  $\phi_0$ , discriminator network parameters  $\beta_0$ , demonstration trajectory  $\tau^E \sim \pi^E$

**Output**: Learned individual policy  $\pi_{\theta, \varphi}$

- 1 Randomly initialize network parameters  $\theta_0, \phi_0, \beta_0$ ;
- 2 Execute initial policy  $\pi_{\theta_0, \phi_0}$ , generate trajectory set  $\xi_0$ , and store  $\xi_0$  in the generated data buffer;
- 3 **for**  $i = 0, 1, 2, \dots, M$  **generation do**
- 4   Sample  $\tau$  from the generated data buffer, sample  $\tau_E$  from demonstration data;
- 5   Update discriminator parameters through gradient ascent using generated samples and expert samples:  
 $\beta_{i+1} \leftarrow \beta_i + \eta_\omega \hat{\mathbb{E}}_\tau [\nabla_\beta \log(D_\beta(s, a))] + \hat{\mathbb{E}}_{\tau_E} [\nabla_\beta \log(1 - D_\beta(s, a))] - \lambda \mathcal{L}_{GP}$ ;
- 6   Optimize pairwise interaction network parameters using an evolutionary strategy algorithm:  $\theta_{i+1}, \xi_1 \leftarrow$   
   OpenAI-ES( $\theta_i, \phi_i, D_{\beta_i}$ );
- 7   Optimize aggregation network parameters using an evolutionary strategy algorithm:  $\varphi_{i+1}, \xi_2 \leftarrow$   
   OpenAI-ES( $\theta_{i+1}, \varphi_i, D_{\beta_i}$ );
- 8   Store generated data  $\xi_1, \xi_2$  in the generated data buffer;
- 9 **end**

---

$\mathcal{L}_{GP} = \mathbb{E}_{s, a \sim \zeta} \left[ \left( \|\nabla_{s, a} D_\beta(s, a)\| - 1 \right)^2 \right]$  is the gradient penalty term, and  $\lambda$  is a coefficient.  $\zeta$  refers to the distribution of data points used for applying the gradient penalty regularization term. This corresponds to uniformly sampling data points along the line connecting samples generated by the policy and demonstration samples. Taking action  $a$  as an example,  $a = ua' + (1-u)a''$ , where  $a' \sim \pi_\theta$  and  $a'' \sim \pi^E$ . The gradient penalty serves to enforce the output function of the discriminator to satisfy the local Lipschitz continuity condition. This guarantees that the difference in output between adjacent samples in the input space does not exceed a fixed upper bound. Literature [34] provides a theoretical analysis demonstrating that the Lipschitz continuity of the discriminator is a necessary condition for achieving stable training in adversarial imitation learning.

## 2. Reward function of the policy network

The policy is updated based on a reward function  $r$ , which is a function of the discriminator's evaluation of the generated data  $D_\beta(s, a)$ . Here,  $D_\beta(s, a)$  represents the probability that the generated data is considered demonstration data, reflecting the similarity between the generated and demonstration data. In the framework of adversarial imitation learning,  $r$  can take various forms, such as the positive logarithmic reward  $r(s, a) = -\log(1 - D_\beta(s, a)) \in (0, \infty)$  and the negative logarithmic reward  $r(s, a) = \log(D_\beta(s, a)) \in (-\infty, 0)$ . Studies have shown that these reward functions may exhibit reward bias compared to the underlying true reward function [35–37]. To mitigate reward bias, a neutral reward function

is employed in this paper:

$$r(s, a) = \log(D_\beta(s, a)) - \log(1 - D_\beta(s, a)) \in (-\infty, \infty) \quad (8)$$

This reward function has been shown to mitigate termination and survival biases [35], and it has been employed in various applications of adversarial imitation learning [38, 39].

(3) Cooperative co-evolution of the pairwise interaction network and the aggregation network

Optimizing the pairwise interaction network and the aggregation networks separately can be seen as a cooperative co-evolution framework based on a "divide and conquer" strategy. By decomposing a high-dimensional problem into subproblems and optimizing each subproblem separately, the cooperative co-evolution framework can improve the optimization efficiency and performance. The cooperative co-evolution framework has been widely applied to large-scale optimization problems such as neural network parameter optimization [40–42]. The deep attention network used in this paper consists of two well-defined modules that can be naturally divided into two parameter vectors  $\phi$  and  $\varphi$ . By optimizing these two networks separately, each can concentrate on its designated task, thereby enhancing its specialization. In addition, by decomposing the problem into two subproblems, the complexity of the overall optimization problem can be reduced. This facilitates improving the convergence speed of optimization.

## Optimization of network parameter with evolutionary strategy

Evolution Strategies (ES) [43] is a class of black-box optimization algorithms inspired by natural evolution. Its search process includes the following steps: (1) Sampling: applying different perturbations (mutations) to a parameter vector (parent) to generate the next generation (population); (2) Evaluation: evaluating the objective function of the population; (3) Recombination: selecting parameter vectors with higher scores and recombining them to form the parent of the next generation. Each execution of the above process is termed as a generation. The process iterates until a predetermined number of generations is reached or the objective function value converges. Compared to gradient-based methods, population-based evolutionary strategies have stronger exploration capabilities in the parameter space and exhibit greater robustness with respect to hyperparameters [44]. In addition, ES algorithms do not require the Markov property in problem formulation, making them less susceptible to challenges posed by non-stationary environments. This advantage makes ES particularly suitable for optimizing individual models in swarm systems.

In this paper, an ES variant proposed by OpenAI, referred to as OpenAI-ES [33], is utilized. OpenAI-ES represents the population as a probability distribution of the parameter vector  $\theta$ , which can be expressed as  $p_\phi(\theta)$ . OpenAI-ES aims to optimize  $\phi$  to maximize the average objective function value of the population:

$$\max \mathbb{E}_{\theta \sim p_\phi} [f(\theta)] \quad (9)$$

where  $f(\theta)$  represents the reward obtained by the policy  $\pi_\theta$  in one simulation, and  $\theta$  denotes the parameters of the policy.

At generation  $t$ , OpenAI-ES samples  $n$  offspring by adding Gaussian noise to the parameter  $\theta_t$ :  $\theta_t^i = \theta_t + \sigma \epsilon_i$ , where  $\epsilon_i \sim \mathcal{N}(0, I)$ . This is equivalent to sampling  $n$  policy parameters  $\theta_t$  from a Gaussian distribution  $p_{\phi_t} \sim \mathcal{N}(\theta_t, \sigma^2 I)$  with mean  $\theta_t$  and variance  $\sigma^2 I$ . Subsequently, simulations are conducted for each sampled parameter  $\theta_t^i$  using the policy  $\pi_{\theta_t^i}$  to obtain rewards  $f(\theta_t^i)$ . Finally, the parameter  $\theta_t$  is updated using the gradient of Eq. (9). Similar to the classical reinforcement learning method REINFORCE [45], this gradient can be estimated using the following formula:

$$\nabla_\phi \mathbb{E}_{\theta \sim p_\phi} [f(\theta)] \approx \frac{1}{n} \sum_{i=1}^n f(\theta_t^i) \nabla_\phi \log p_\phi(\theta_t^i) \quad (10)$$

Substituting the Gaussian distribution  $p_{\phi_t} \sim \mathcal{N}(\theta_t, \sigma^2 I)$  into Eq. (10) and employing gradient ascent, the update formula for policy parameters  $\theta$  is given by:

$$\theta_{t+1} \leftarrow \theta_t + \frac{\lambda}{n\sigma^2} \sum_{i=1}^n f(\theta_t^i) \epsilon_i \quad (11)$$

where  $\lambda$  is the learning rate.

The completed process of optimizing network parameters using OpenAI-ES is shown in Algorithm 2. In this paper, we utilize a dual sampling technique [46, 47] to decrease the variance of gradient estimation. The perturbation vectors generated by the dual sampling method are symmetry, which helps mitigate potential biased searches. Moreover, the dual sampling method yields identical samples with only half the original sampling times, thus improving operational and storage efficiency. The steps of dual sampling are as follows: initially,  $n$  perturbation vectors  $\epsilon_i$  are sampled from  $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ , followed by the generation of positive and negative perturbations,  $\theta_{i+} = \theta + \epsilon_i$  and  $\theta_{i-} = \theta - \epsilon_i$  (step 2). Subsequently, collective behavior simulations are executed for policies  $\pi_{\theta_{i+}}$  and  $\pi_{\theta_{i-}}$ , where all individuals in the swarm employ the same policy (step 3–4, step 6–7). The trajectories  $\tau^+$  and  $\tau^-$  generated by simulations are stored, and rewards  $R^+$  and  $R^-$  are obtained by evaluating the generated trajectories using the discriminator (step 5, step 8). Finally, network

parameters are updated based on Eq. (12), and then the learning rate and the perturbation sampling standard deviation are adjusted accordingly (step 10–13).

$$\bar{\theta}_{t+1} \leftarrow \bar{\theta}_t + \lambda \frac{1}{2\sigma^2 n} \sum_{i=1}^{2n} \bar{\epsilon}_i (R_i^+ - R_i^-) \quad (12)$$

---

#### Algorithm 2: Optimizing Network Parameters with OpenAI-ES

---

**Input** : Network parameters  $\theta_0$ , Discriminator network  $D_\beta$ , Learning rate  $\lambda$ , Perturbation standard deviation  $\sigma$

**Output**: Optimized network parameters  $\theta_M$ , Generated trajectory set  $\xi = \{\tau_1^+, \dots, \tau_n^+, \tau_1^-, \dots, \tau_n^-\}$

---

```

1 for  $j = 1, \dots, n$  random perturbations do
2   Sample  $\epsilon_j \leftarrow \sigma \mathcal{N}(0, I)$ ;
3   Reset environment to random initialization  $S_{i,j}$ ;
4   Sample trajectory  $\tau_j^+$  using policy parameters  $\theta^{i-1} + \epsilon_j$ ;
5   Obtain trajectory reward
      $R_j^+ = \log(D_\beta(\tau_j^+)) - \log(1 - D_\omega(\tau_j^+))$  from the
     discriminator;
6   Reset environment to previous initialization  $S_{i,j}$ ;
7   Sample trajectory  $\tau_j^-$  using policy parameters  $\theta^{i-1} - \epsilon_j$ ;
8   Obtain trajectory reward
      $R_j^- = \log(D_\beta(\tau_j^-)) - \log(1 - D_\omega(\tau_j^-))$  from the
     discriminator;
9 end
10 Normalize  $\{R_j\}_{1,\dots,n}$ ;
11  $\bar{\omega}^i \leftarrow \bar{\omega}^{i-1} + \lambda \frac{1}{2\sigma^2 n} \sum \bar{\epsilon}_j (R_j^+ - R_j^-)$ ;
12 Adjust learning rate:  $\lambda \leftarrow \gamma \lambda$ ;
13 Adjust perturbation standard deviation:  $\sigma \leftarrow \gamma \sigma$ ;
```

---

## Experiments

In this section, we imitated three different collective behaviors to validate the effectiveness of the proposed algorithm. The algorithm parameter settings are detailed in the appendix.

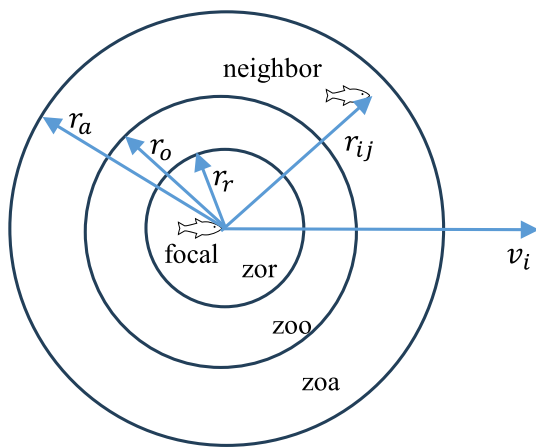
### Collective behavior simulation model

The collective behavior simulation in this paper involves a 2D environment, where individuals possess four attributes:  $x$  coordinate,  $y$  coordinate, velocity  $v$ , and heading angle  $\theta \in [0, 2\pi]$ . The position update formula for individuals is

$$x(t+1) = x(t) + v * \cos \theta(t) \quad (13)$$

$$y(t+1) = y(t) + v * \sin \theta(t) \quad (14)$$

In this paper, the individual's velocity  $v$  is fixed at 1. Individuals modify their state by adjusting their heading angle.



**Fig. 4** The schematic diagram of the Couzin model

The individual's heading angle  $\theta$  is updated based on the individual action  $a$ :

$$\theta(t+1) = \theta(t) + a_{\max}^E 2 \left( a - \frac{1}{2} \right) \quad (15)$$

Here,  $a_{\max}^E = \max |\Delta\theta|$  represents the maximum turning in a single step in the demonstration data. Since  $a \in [0, 1]$ , the individual action is transformed to the interval  $[-a_{\max}^E, a_{\max}^E]$  through  $a_{\max}^E 2(a - \frac{1}{2})$ .

### Demonstration data—three typical collective behaviors from the Couzin model

We selected three typical and distinct collective behavior trajectories generated by the Couzin model as the demonstration data. The Couzin model [48] is a mathematical model used to describe collective behaviors. This model focuses on simulating the movement and interactions of individuals within animal swarms, typically fish schools or bird flocks. A key feature of this model is that the response of a focal individual  $i$  to its neighbors is guided by the expected direction  $D(t)$ . The  $D(t)$  is calculated using the state of neighbors. Individual  $i$  computes its own  $D_i(t)$  and turns towards it at a constant turning rate.

In the Couzin model, the perceptual space of the focal individual is divided into three non-overlapping regions: the zone of repulsion (zor), the zone of orientation (zoo), and the zone of attraction (zoa), as shown in Fig. 4. The radii of these three regions are denoted as  $r_r$ ,  $r_o$ , and  $r_a$ , respectively. The expected direction  $D_i(t)$  is calculated based on the neighbor states in every region. The  $n_r$ ,  $n_o$ , and  $n_a$  represent the number of neighbors in the repulsion zone, orientation zone, and attraction zone, respectively. There are three ways to compute  $D_i(t)$ :

1. If there are neighbors in the repulsion zone ( $n_r > 0$ ), then

$$D_i(t) = - \sum_{j \neq i}^{n_r} \frac{\mathbf{r}_{ij}(t)}{|\mathbf{r}_{ij}(t)|} \quad (16)$$

where  $\mathbf{r}_{ij}$  represents the vector from the focal individual  $i$ 's position to the neighbor  $j$ 's position. This formula indicates that the focal individual's expected direction of motion is the opposite of the relative position vector of neighbors in the repulsion zone. In other words, the focal individual tends to move away from neighbors in the repulsion zone.

2. If there are no neighbors in the repulsion zone ( $n_r = 0$ ), but there are neighbors in either the orientation zone or the attraction zone ( $n_o > 0$  or  $n_a > 0$ ), then  $D_i(t)$  is defined as:

$$D_i(t) = \sum_{j=1}^{n_o} \frac{\mathbf{v}_j(t)}{|\mathbf{v}_j(t)|} + \sum_{j=1}^{n_a} \frac{\mathbf{r}_{ij}(t)}{|\mathbf{r}_{ij}(t)|} \quad (17)$$

The first term represents the sum of neighbor velocity directions in the orientation zone, which aligns the focal individual with the directions of neighbors in the orientation zone. The second term represents the sum of relative position vectors to neighbors in the attraction zone, exerting an attraction force on the focal individual towards its neighbors.

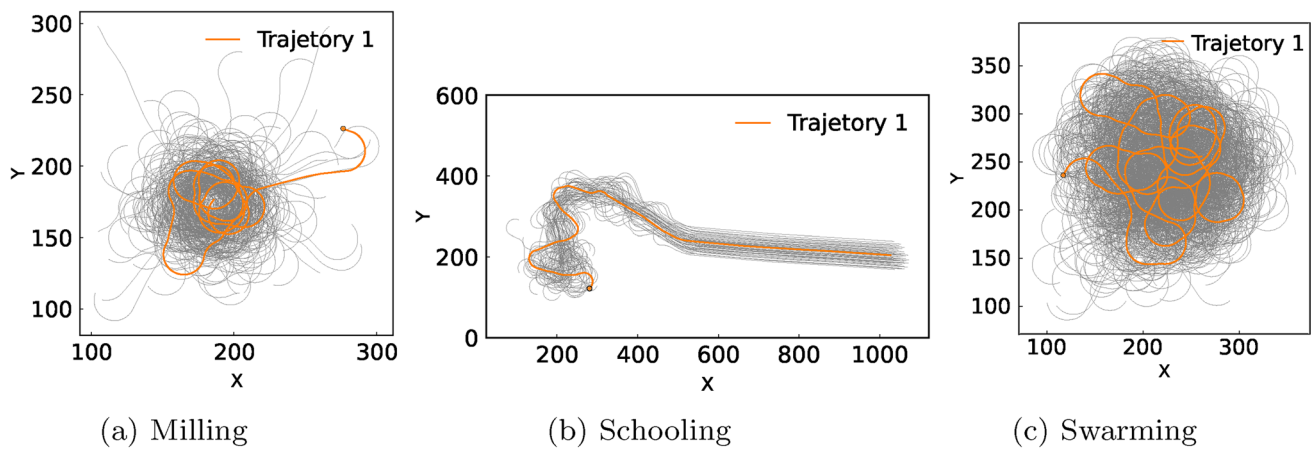
(3) Otherwise,  $D_i(t) = \mathbf{v}_i(t)$ , meaning the current velocity orientation is maintained.

By adjusting  $r_r$ ,  $r_o$ , and  $r_a$ , the Couzin model can exhibit three types of behaviors: milling where individuals perform circular motions around a common center, schooling where all individuals move in the same direction, and swarming with no correlation in velocity direction among individuals. The simulated trajectories of these three typical collective behaviors are shown in Fig. 5. These three trajectories are used as demonstration data for imitation learning in this paper. Each trajectory consists of 1000 steps and includes 50 individuals.

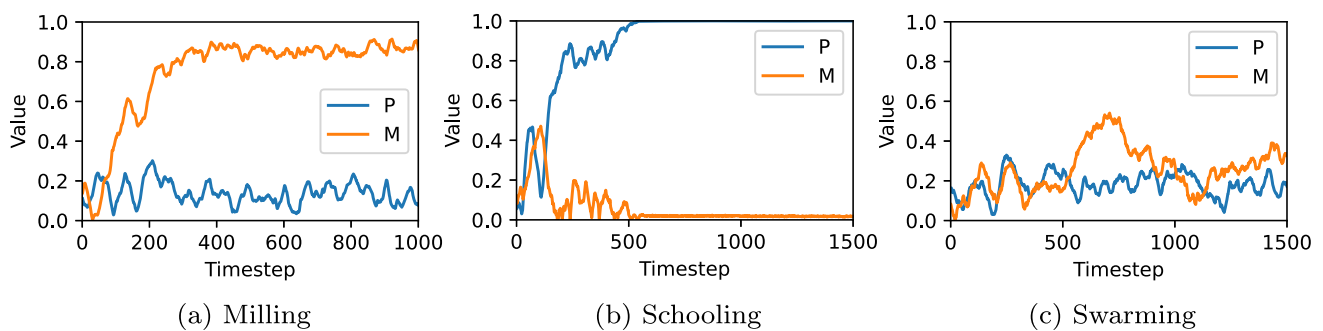
### Metrics of collective behavior

The polarization and angular momentum [48] are commonly used metrics in the study of collective behavior. They serve to reveal the overall movement patterns of collective behaviors. The polarization (hereinafter referred to as  $p$ -value) is a measure of the coherence of movement within the swarm. It reflects the degree of consistency in the movement directions of individuals in the swarm. The formula for polarization is as follows:

$$p = \left| \frac{1}{N} \sum_{i=1}^N \mathbf{v}_i \right| \quad (18)$$



**Fig. 5** Collective behavior trajectories based on the Couzin model (demonstration data)



**Fig. 6** The  $p$ -values and  $m$ -values of the demonstration data in Fig. 5

where  $N$  represents the number of individuals in the swarm, and  $\mathbf{v}_i$  denotes the velocity vector of individual  $i$ .

The angular momentum (hereinafter referred to as the  $m$ -value) describes the rotation and directional changes of a swarm. The formula for  $m$ -value is as follows:

$$m = \left| \frac{1}{N} \sum_{i=1}^N \mathbf{r}_{ic} \times \mathbf{v}_i \right| \quad (19)$$

Here,  $\mathbf{r}_{ic}$  represents the vector from the individual  $i$  to the swarm center, and  $\times$  denotes the cross-product operation between vectors.

Figure 6 illustrates the  $p$ -value and  $m$ -value of the milling, schooling, and swarming behaviors. These three behaviors exhibit distinct metrics. The milling behavior demonstrates high  $m$ -values but low  $p$ -values. In contrast, schooling behavior shows high  $p$ -values but low  $m$ -values. The swarming behavior exhibits low values for both  $p$  and  $m$ .

### Results for imitating milling behavior

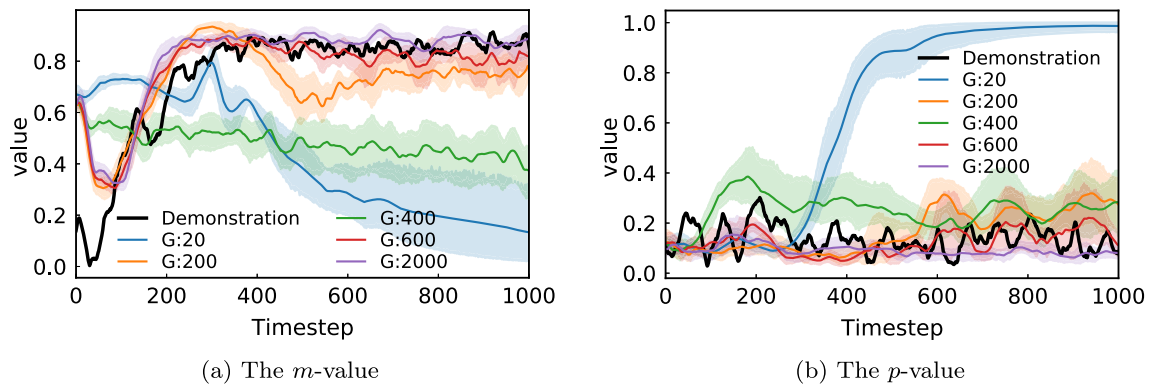
The models learned during the training process are saved in and these models are sampled 10 times with 1000 steps each. The  $p$ -value and  $m$ -value of these models compared with

demonstration data are presented in Fig. 7. The  $G$  represents the generation of policy optimization. From Fig. 7a, it can be seen that the  $m$ -value of the models is gradually increasing with the optimization process, and from Fig. 7b, it can be seen that the  $p$ -value is gradually decreasing with the optimization process. Figure 8 shows the trajectories (Fig. 8a) and statistical metrics (Fig. 8b) produced by the imitated model. The statistical metrics show that the imitation model exhibits behavior consistent with the high  $m$ -value and low  $p$ -value that characterize the milling behavior in the demonstration data. This indicates that the imitation learning method proposed in this paper is effective and the model achieves the milling behavior demonstrated in the demonstration data.

### Results for imitating schooling behavior

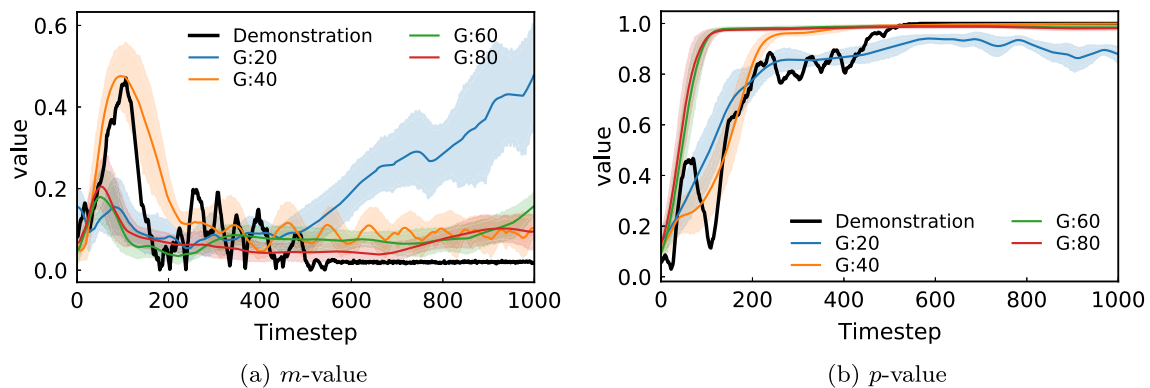
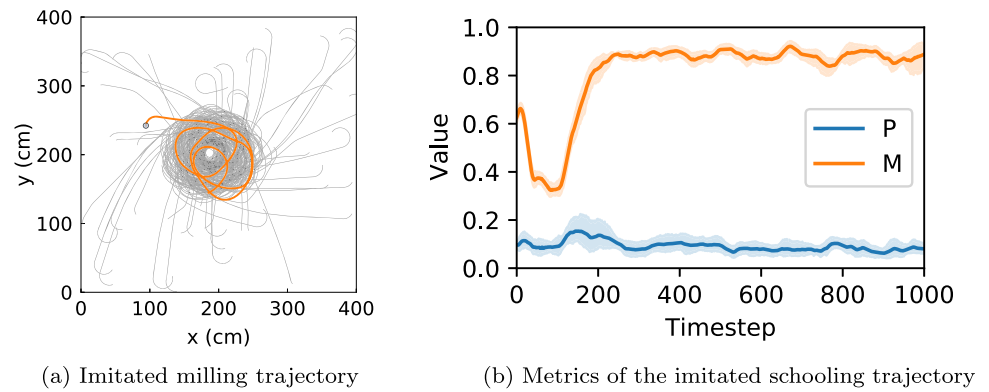
Schooling behavior is characterized by a high  $p$ -value and low  $m$ -value. Figure 9 illustrates the imitation process of schooling behavior. It can be observed from the figure that, as the optimization progresses, the  $m$ -value of the trajectory of the schooling behavior decreases (Fig. 9a), while the  $p$ -value increases (Fig. 9b).

Figure 10 presents the trajectories of collective behavior generated by the model in the 80-th generation and the



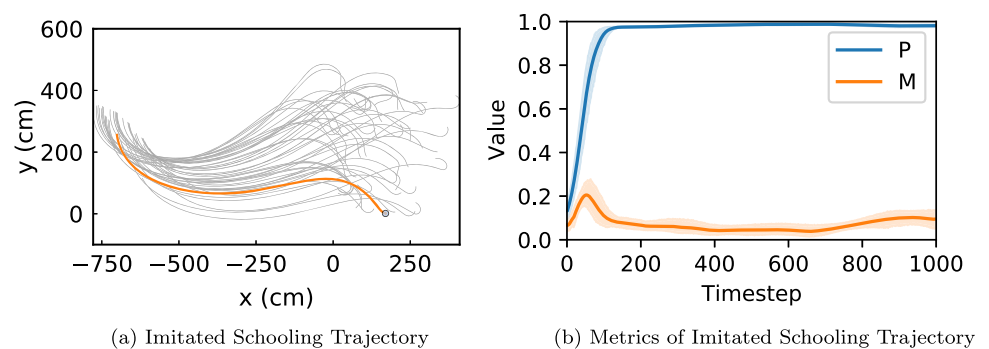
**Fig. 7** Imitation process of the milling behavior

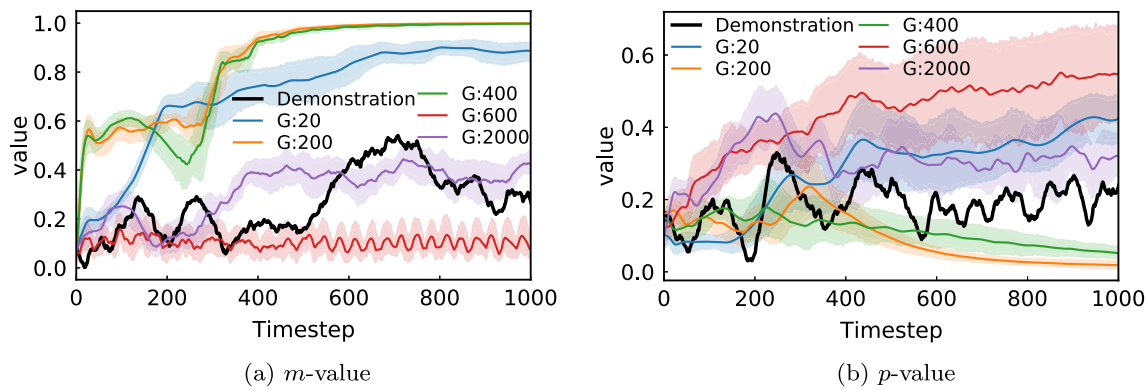
**Fig. 8** Imitation results of the milling behavior



**Fig. 9** Imitation process of schooling behavior

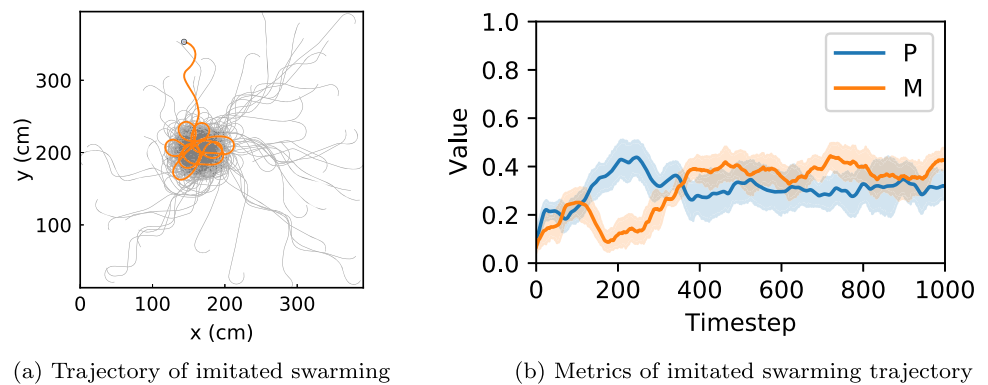
**Fig. 10** Imitation results of schooling behavior





**Fig. 11** Imitation process of swarming behavior

**Fig. 12** Imitation results of swarming behavior



corresponding statistical metrics. The trajectories demonstrate directional consistency, and the metrics align with the characteristics of schooling behavior. This indicates that the proposed method in this paper successfully imitates the schooling behavior in the demonstration data. It is noteworthy that the method proposed in this paper only requires 80 generations to exhibit schooling behavior, indicating that schooling behavior is relatively simple.

## Results for imitating swarming behavior

Swarming behavior is characterized by the low  $p$ -value and low  $m$ -value. The swarm aggregates within a region, with motion lacking directional coherence and significant rotation. Figure 11 depicts the imitation process of swarming behavior, showing that around the 2000-th generation, the  $p$ -value is approximately 0.3 and the  $m$ -value is around 0.4, which closely aligns with the demonstration data. Figure 12 showcases the trajectories of the model in the 2000-th generation (Fig. 12a) and the metrics obtained from 10 simulation runs (Fig. 12b). It can be observed from the figures that the learning method proposed in this paper successfully imitates swarming behavior, exhibiting the characteristic of low  $p$ -value and low  $m$ -value as well.

## Comparison with the behavior cloning method

We implemented a behavior cloning method with the same network architecture as the proposed method, using the same demonstration data. This method is trained for 100 epochs, using the MSE loss function, the Adam optimizer, a batch size of 512, and a learning rate of 0.001.

We have conducted comparative experiments with this behavior cloning method on the milling behavior. The milling behavior is characterized by high  $m$  values and low  $p$  values. Figure 13 shows the mean of the  $m$  and  $p$  values of the 10 trajectories (each of 1000 steps) generated by the learned models, under different amounts of demonstration data. As can be seen from the Fig. 13, compared with the behavior cloning method, our method achieved  $m$  and  $p$  values that are closer to the demonstration data. Furthermore, our proposed method ensures stable performance with fewer demonstration data, whereas the behavior cloning method exhibited a significant performance decline. This is because the behavior cloning algorithm is more susceptible to compounding error, requiring more demonstration data. In contrast, the AIL method encourages policies to explicitly match the expert state-action distribution by exploring in the policy space, enabling the discovery of better policies.

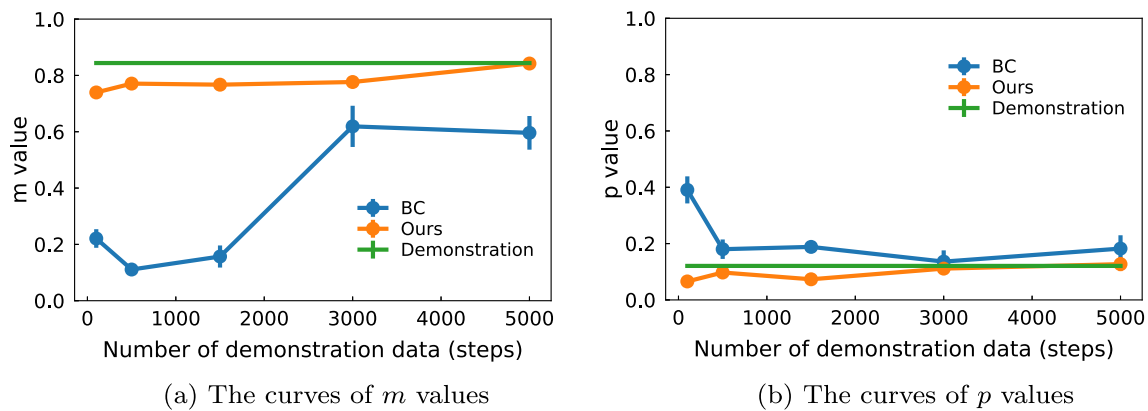


Fig. 13 Comparison with BC on different amounts of demonstration data

### Exploring the diversity of individual policies within collective behavior

The individual policies implicit in collective behaviors are not unique. For example, in the Couzin model, the parameters that generate specific collective behavior are not unique but exist within a parameter space [48]. Within this parameter space, various parameters can yield nearly identical collective behavior. In adversarial imitation learning methods, the generator explores the strategy space to deceive the discriminator. Through multiple experiments, we found that our approach can uncover different individual policy models hidden in the demonstration data. Our research thus provides inverse verification that the individual policies underlying collective behaviors are not unique.

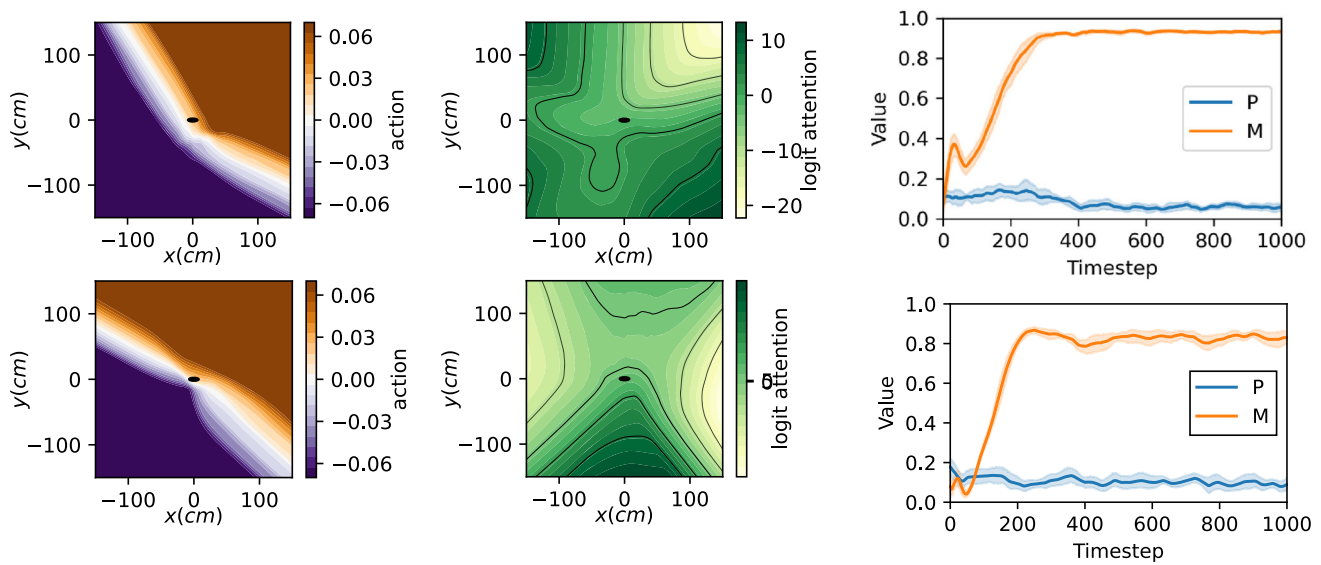
In this section, taking advantage of the modularity of deep attention networks, the input–output mappings of the pairwise interaction network and the aggregation networks, are analyzed respectively, to understand the diversity of individual policies for collective behaviors. Take the two individual policies obtained by imitating the milling behavior as an example. The first column of Fig. 14 shows the mapping between the neighbor position and the focal individual action in the pairwise interaction network. The focal action is the average with respect to a uniform distribution over all possible relative neighbor orientations (from 0 to  $2\pi$ ). This figure depicts the response of the focal individual to neighbors at different locations. The second column of Fig. 14 shows the mapping between neighbor positions and aggregation weights in the aggregation network. Again, the weight is the average with respect to a uniform distribution over all possible relative neighbor orientations (from 0 to  $2\pi$ ). This figure depicts the weights of neighbors at different locations on the focal individual. The third column of Fig. 14 shows the curves of the  $p$ -value and  $m$ -value of the trajectories generated by simulating the model ten times.

It can be observed from the figure that although the collective behaviors generated by both models exhibit characteristics of high  $m$ -value and low  $p$ -value, they differ in their individual policies. By comparing two pairwise interaction networks, we can observe that although both maintain their own direction ( $a \approx 0$ ) when neighbors are positioned to the right front, there exists a difference in the width of this region. In terms of the aggregation network, although both models exhibit higher weights for neighbors to the right than to the left, there is a significant difference in the weights between the front and rear directions. The reason for the higher weights on the right side may be because the swarm is rotating clockwise, resulting in the majority of neighbors being on its right side, thereby directing attention towards the right side.

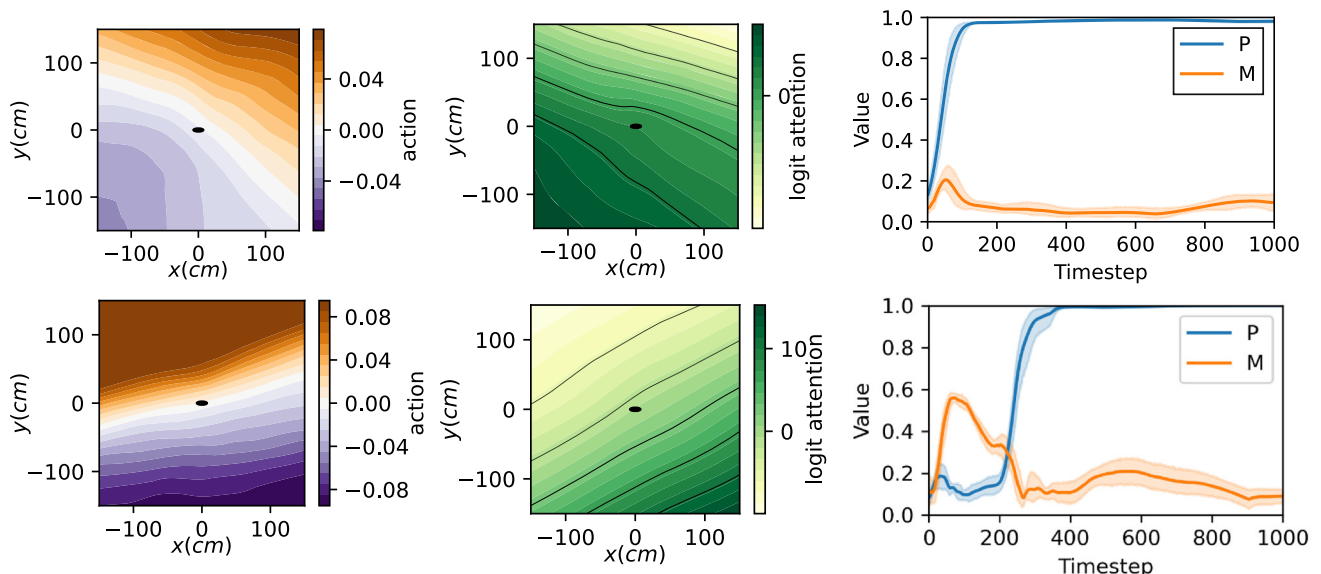
Figure 15 illustrates two distinct schooling policies obtained by the proposed algorithm, while Fig. 16 presents two swarming policies obtained by the proposed algorithm. From these figures, it can be observed that the imitation learning method proposed in this paper has discovered different individual policies, all of which are capable of manifesting the collective behaviors present in the demonstration data. This indicates that the adversarial imitation learning approach employed in this paper possesses a certain level of exploratory capability, enabling the discovery of diverse policies hidden within the demonstration data.

### Experiments on swarm robot

This section applies the policies imitated from the demonstration data to swarm robots. The aim is to validate the feasibility of designing control policies for swarm robots using the proposed imitation learning method.



**Fig. 14** Two milling policies obtained by the proposed algorithm



**Fig. 15** Two schooling policies obtained by the proposed algorithm

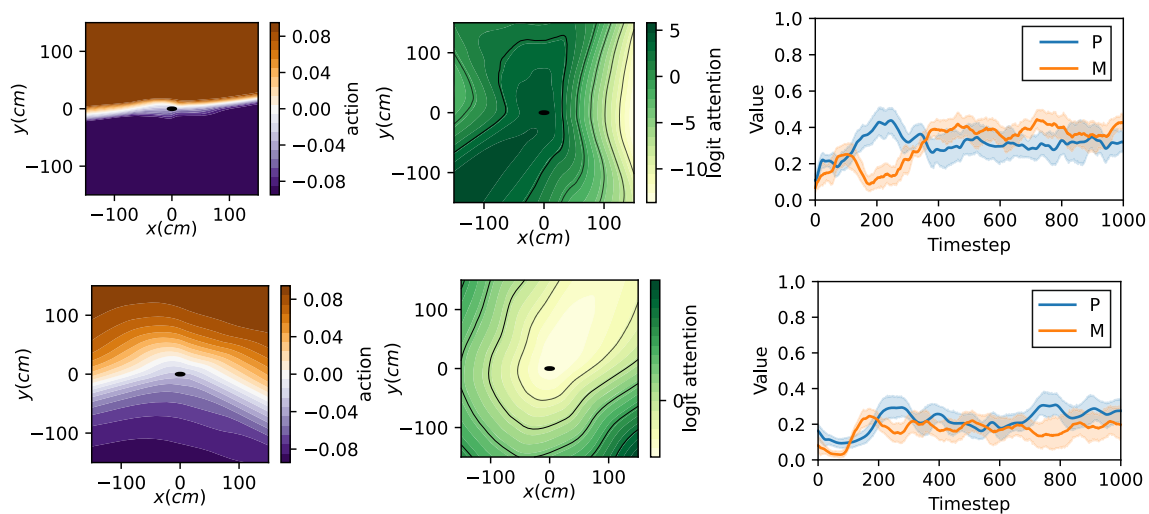
### Swarm robots experiment platform

We validate the imitated swarm models using a self-developed small-wheeled swarm robot experimental system. The system comprises many small-wheeled robots (see Fig. 17), a computer, a motion capture system, a wireless communication system, and an industrial camera image acquisition system (see Fig. 17). The small-wheeled robots utilize differential drive motion and possess wireless communication and motion control capabilities, receiving real-time motion commands from the host computer. The motion capture system captures the position information of the robots, which is then input to the computer. The swarm models make

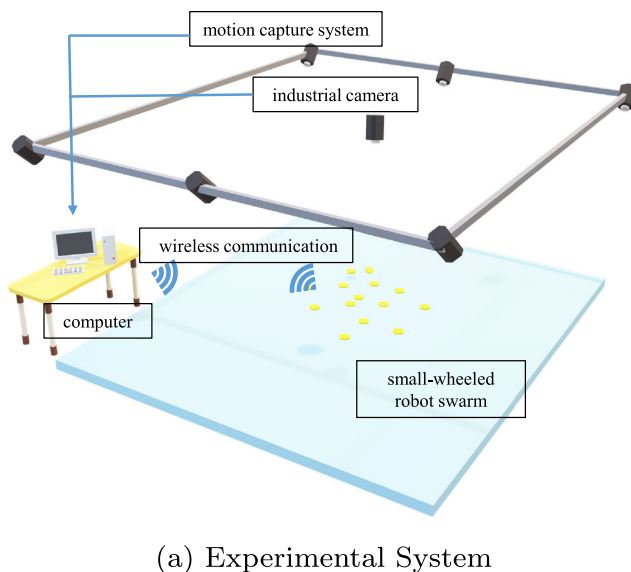
decisions and compute the next actions for each robot based on the neighbor information of each robot. The host computer sends motion commands to the robots via wireless communication for execution.

### Results of experiments on swarm robot

We implement the milling, schooling, and swarming policies obtained by the algorithm proposed in this paper on a robot swarm, respectively. In each experiment, 20 robots are used. Figures 18, 19, and 20 respectively illustrate the trajectories and statistical metrics of the robot swarm executing milling, schooling, and swarming policies. In Fig. 18,



**Fig. 16** Two swarming policies obtained by the proposed algorithm



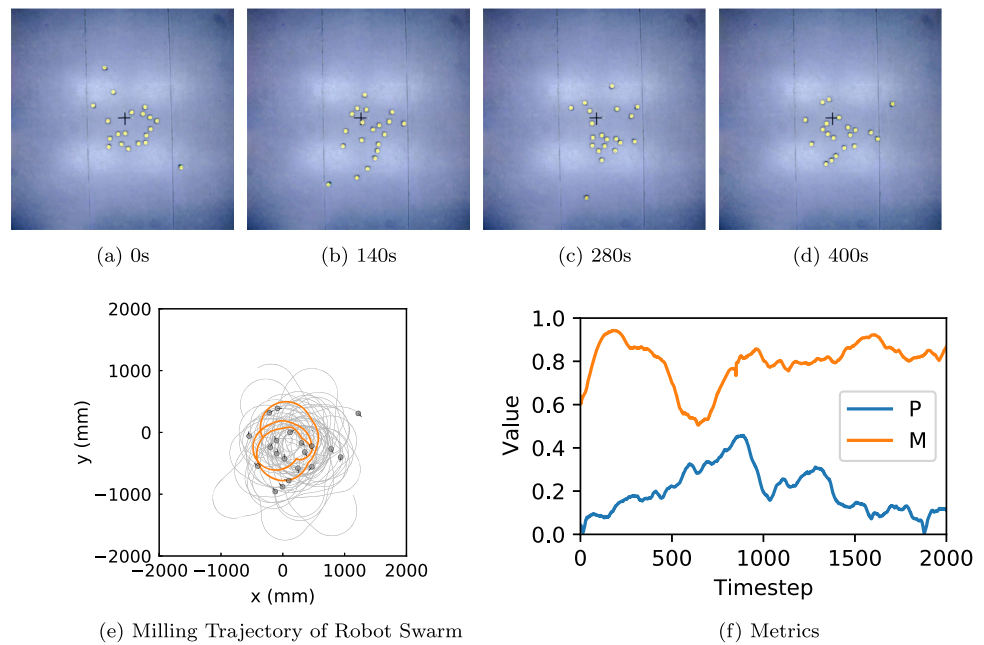
**Fig. 17** Swarm robot experiment system

it can be observed that the robot swarm exhibits a vortex shape, characterized by a low  $p$ -value and high  $m$ -value. Figure 19 reveals that the robot swarm demonstrates coherence in motion direction, accompanied by high  $p$ -value and low  $m$ -value. Figure 20 shows the robot swarm gathering in a specific area, displaying disordered movement patterns with relatively low  $p$ -value and  $m$ -value. On the robot swarm, the behaviors of the three policies obtained through imitation closely resemble the demonstration data and exhibit identical statistical metrics. Through experiments conducted on the robot swarm, we validate that policies learned by imitation learning from collective behavior demonstration can be applied to control swarm robots.

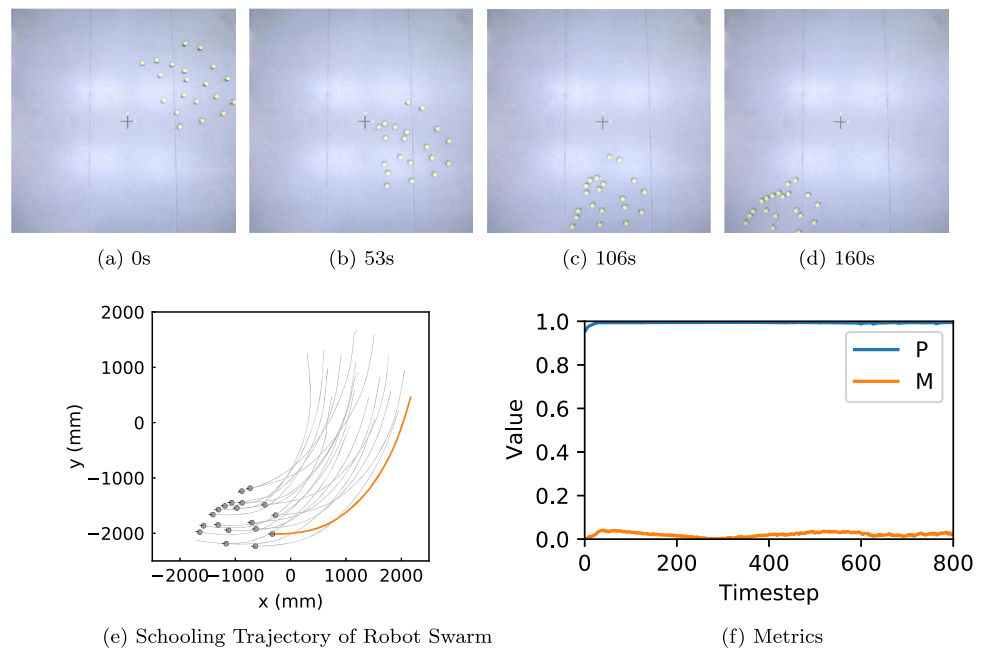
## Conclusion

Recovering individual policies from collective behavior contributes to a deeper understanding of biological swarm systems and guides the design of control strategies for swarm robotics. Leveraging an adversarial imitation learning framework, we propose a method utilizing the deep attention network to imitate collective behavior. By setting up the Couzin model, we generated three distinct collective behaviors—milling, schooling, and swarming—as demonstration data. Our approach successfully imitated these three collective behaviors. Through the analysis of individual policies derived from the algorithm, we verified that the indi-

**Fig. 18** Experiment on swarm robots with imitated milling policy



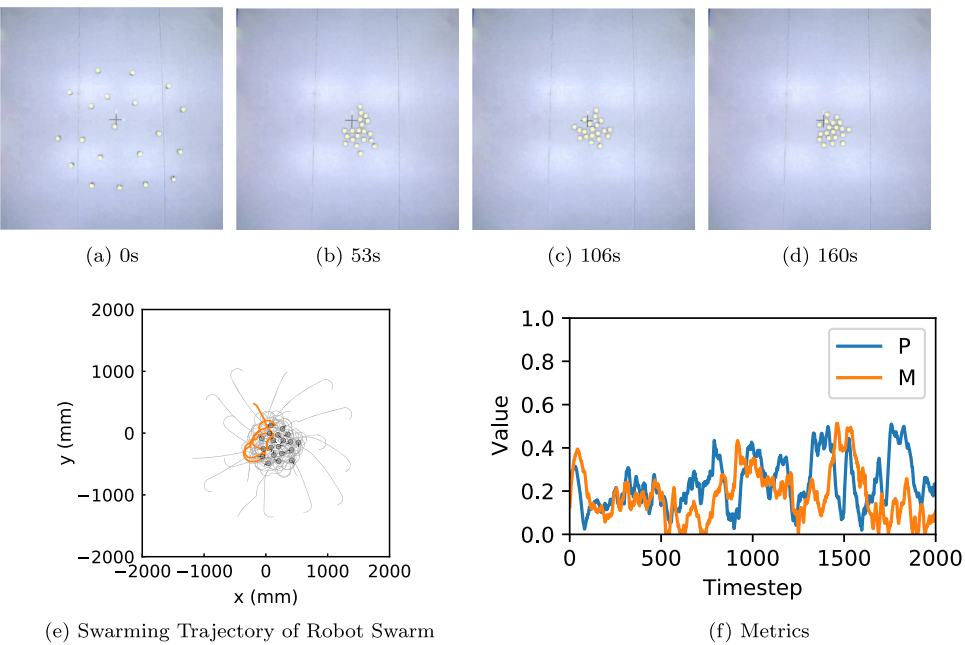
**Fig. 19** Experiment on swarm robots with imitated schooling policy



vidual policies underlying a certain collective behavior are not unique. Finally, we applied the learned individual policies to control a robot swarm. Experimental results demonstrate that the policies obtained by our algorithm can exhibit behaviors similar to the demonstration data on the robot swarm.

In the future, a potential research direction is to employ quality-diversity methods to illuminate the property space of a specific collective behavior. This may help us understand the complexity of swarm systems.

**Fig. 20** Experiment on swarm robots with imitated swarming policy



Appendix A. Parameter setting of the proposed algorithm

See Table 1.

**Table 1** Parameter setting of the proposed method

| Parameter description                                         | Value                            |
|---------------------------------------------------------------|----------------------------------|
| Pairwise network structure                                    | MLP with size [4 × 100 × 30 × 2] |
| Aggregation network structure                                 | MLP with size [4 × 100 × 30 × 1] |
| Activate function                                             | ReLU                             |
| The population size of OpenAI-ES                              | 30                               |
| Discriminator optimizer                                       | RMSProp                          |
| The annealing rate $\gamma$ in OpenAI-ES                      | 0.9997                           |
| Initial learning rate $\lambda$ in OpenAI-ES                  | 0.1                              |
| Initial perturbation standard deviation $\sigma$ in OpenAI-ES | 0.1                              |

**Funding** This research was funded by the National Natural Science Foundation of China under Grants 62076203 and in part by the Fundamental Research Funds for the Central Universities under Grant G2024KY0601.

**Data availability** The relevant data can be obtained by contacting the first author.

Declarations

**Conflict of interest** On behalf of all authors, the corresponding author states that there is no Conflict of interest.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Lopez U, Gautrais J, Couzin ID, Theraulaz G (2012) From behavioural analyses to models of collective motion in fish schools. *Interface focus* 2(6):693–707
2. Sokolov A, Aranson IS, Kessler JO, Goldstein RE (2007) Concentration dependence of the collective dynamics of swimming bacteria. *Phys Rev Lett* 98(15):158102
3. Zou H, Su H, Song S, Zhu J (2018) Understanding human behaviors in crowds by imitating the decision-making process. In: *Proceedings of the AAAI conference on artificial intelligence*, vol 32
4. Vriend NJ (1995) Self-organization of markets: an example of a computational approach. *Comput Econ* 8:205–231
5. Scharre P (2018) How swarming will change warfare. *Bull Atom Sci* 74(6):385–389
6. Albiero D, Garcia AP, Umezú CK, de Paulo RL (2022) Swarm robots in mechanized agricultural operations: a review about challenges for research. *Comput Electron Agric* 193:106608
7. Duarte M, Gomes J, Costa V, Rodrigues T, Silva F, Lobo V, Marques MM, Oliveira SM, Christensen AL (2016), Application of

- swarm robotics systems to marine environmental monitoring. In: OCEANS 2016-Shanghai, IEEE, pp 1–8
8. Hsu A, Wong-Foy A, McCoy B, Cowan C, Marlow J, Chavez B, Kobayashi T, Shockey D, Pelrine R (2016) Application of micro-robots for building carbon fiber trusses. In: International conference on manipulation, automation and robotics at small scales (MARSS). IEEE 2016, pp 1–6
  9. Zheng Z, Lei X, Peng X (2022) Selective interaction and its effect on collective motion. *Sci Rep* 12(1):8601
  10. Hüttenrauch M, Šošić A, Neumann G (2019) Deep reinforcement learning for swarm systems. *J Mach Learn Res* 20(54):1–31
  11. Wang T, Peng X, Wang T, Liu T, Xu D (2024) Automated design of action advising trigger conditions for multiagent reinforcement learning: a genetic programming-based approach. *Swarm Evol Comput* 85:101475
  12. Bezcioglu M. B, Lennox B, Arvin F (2021) Self-organised swarm flocking with deep reinforcement learning. In: 2021 7th International conference on automation, robotics and applications (ICARA), IEEE, pp 226–230
  13. Hasselmann K, Ligot A, Ruddick J, Birattari M (2021) Empirical assessment and comparison of neuro-evolutionary methods for the automatic off-line design of robot swarms. *Nat Commun* 12(1):4345
  14. Kaiser TK, Hamann H (2019) Engineered self-organization for resilient robot self-assembly with minimal surprise. *Robot Auton Syst* 122:103293
  15. Pomerleau DA (1988) Alvin: an autonomous land vehicle in a neural network. In: Proceedings of the advances in neural information processing systems, Vienna, pp 1–20
  16. Ng AY, Russell SJ (2000) Algorithms for inverse reinforcement learning. In: Proceedings of the seventeenth international conference on machine learning, ICML '00, Morgan Kaufmann Publishers Inc., San Francisco, pp 663–670
  17. Zhou S, Phielipp MJ, Sefair JA, Walker SI, Amor HB (2019), Clone swarms: learning to predict and control multi-robot systems by imitation. In: 2019 IEEE/RSJ international conference on intelligent robots and systems (IROS), IEEE, pp 4092–4099
  18. Guo C, Zhu P, Zhou Z, Lang L, Zeng Z, Lu H (2021) Imitation learning with graph neural networks for improving swarm robustness under restricted communications. *Appl Sci* 11(19):9055
  19. Papaspyros V, Escobedo R, Alahi A, Theraulaz G, Sire C, Mondada F (2024) Predicting the long-term collective behaviour of fish pairs with deep learning. *J R Soc Interface* 21(212):20230630
  20. Ross S, Gordon G, Bagnell D (2011) A reduction of imitation learning and structured prediction to no-regret online learning. In: Proceedings of the fourteenth international conference on artificial intelligence and statistics, JMLR Workshop and Conference Proceedings, pp 627–635
  21. Šošić A, KhudaBukhsa WR, Zoubir AM, Koeppl H (2017) Inverse reinforcement learning in swarm systems. In: Proceedings of the 16th conference on autonomous agents and multiagent systems, pp 1413–1421
  22. Yu X, Wu W, Feng P, Tian Y (2021) Swarm inverse reinforcement learning for biological systems. In: 2021 IEEE International conference on bioinformatics and biomedicine (BIBM), IEEE, pp 274–279
  23. Pinsler R, Maag M, Arenz O, Neumann G (2018) Inverse reinforcement learning of bird flocking behavior. In: ICRA swarms workshop
  24. Heras FJ, Romero-Ferrero F, Hinz RC, de Polavieja GG (2019) Deep attention networks reveal the rules of collective motion in zebrafish. *PLoS Comput Biol* 15(9):e1007354
  25. Ho J, Ermon S (2016) Generative adversarial imitation learning. *Proc Adv Neural Inf Process Syst* 29:1–20
  26. Tolstaya E, Gama F, Paulos J, Pappas G, Kumar V, Ribeiro A (2020) Learning decentralized controllers for robot swarms with graph neural networks. In: Conference on robot learning, PMLR, pp 671–682
  27. Gama F, Marques AG, Leus G, Ribeiro A (2018) Convolutional neural network architectures for signals supported on graphs. *IEEE Trans Signal Process* 67(4):1034–1049
  28. Schilling F, Lecoeur J, Schiano F, Floreano D (2019) Learning vision-based flight in drone swarms by imitation. *IEEE Robot Autom Lett* 4(4):4523–4530
  29. Vicsek T, Czirók A, Ben-Jacob E, Cohen I, Shochet O (1995) Novel type of phase transition in a system of self-driven particles. *Phys Rev Lett* 75(6):1226
  30. Romero-Ferrero F, Heras FJ, Rance D, de Polavieja GG (2023) A study of transfer of information in animal collectives using deep learning tools. *Philos Trans R Soc B* 378(1874):20220073
  31. LaChance J, Suh K, Clausen J, Cohen DJ (2022) Learning the rules of collective cell migration using deep attention networks. *PLoS Comput Biol* 18(4):e1009293
  32. Costa T, Laan A, Heras FJ, De Polavieja GG (2020) Automated discovery of local rules for desired collective-level behavior through reinforcement learning. *Front Phys* 8:200
  33. Salimans T, Ho J, Chen X, Sidor S, Sutskever I (2017) Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*
  34. Blondé L, Strasser P, Kalousis A (2022) Lipschitzness is all you need to tame off-policy generative adversarial imitation learning. *Mach Learn* 111(4):1431–1521
  35. Jena R, Agrawal S, Sycara K (2020) Addressing reward bias in adversarial imitation learning with neutral reward functions. *arXiv preprint arXiv:2009.09467*
  36. Orsini M, Raichuk A, Hussenot L, Vincent D, Dadashi R, Girgin S, Geist M, Bachem O, Pietquin O, Andrychowicz M (2021) What matters for adversarial imitation learning? In: Proceedings of the advances in neural information processing systems, vol 34, Virtual Conference, pp 14656–14668
  37. Zhang M, Wang Y, Ma X, Xia L, Yang J, Li Z, Li X (2020) Wasserstein distance guided adversarial imitation learning with reward shape exploration. In: Proceedings of the IEEE 9th data driven control and learning systems conference, IEEE, Liuzhou, pp 1165–1170
  38. Shin W, Kang H, Hong S (2019) Mature gail: imitation learning for low-level and high-dimensional input using global encoder and cost transformation. *arXiv preprint arXiv:1909.03200*
  39. Zuo G, Zhao Q, Huang S, Li J, Gong D (2021) Adversarial imitation learning with mixed demonstrations from multiple demonstrators. *Neurocomputing* 457:365–376
  40. Yang P, Zhang H, Yu Y, Li M, Tang K (2022) Evolutionary reinforcement learning via cooperative coevolutionary negatively correlated search. *Swarm Evol Comput* 68:100974
  41. Liang J, Chen G, Qu B, Yue C, Yu K, Qiao K (2021) Niche-based cooperative co-evolutionary ensemble neural network for classification. *Appl Soft Comput* 113:107951
  42. Peng F, Liu S, Lu N, Tang K (2022) Training quantized deep neural networks via cooperative coevolution. In: International conference on sensing and imaging. Springer, pp 81–93
  43. Hünig A (1976) Evolutionsstrategie. optimierung technischer systeme nach prinzipien der biologischen evolution
  44. Majid AY, Saaybi S, Francois-Lavet V, Prasad RV, Verhoeven C (2023) Deep reinforcement learning versus evolution strategies: a comparative survey. In: IEEE transactions on neural networks and learning systems
  45. Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach Learn* 8:229–256
  46. Brockhoff D, Auger A, Hansen N, Arnold D. V, Hohm T (2010) Mirrored sampling and sequential selection for evolution strategies. In: Parallel problem solving from nature, PPSN XI: 11th

- international conference, Kraków, Poland, September 11–15, 2010, Proceedings, Part I 11, Springer, pp 11–21
47. Chrabaszcz P, Loshchilov I, Hutter F (2018) Back to basics: benchmarking canonical evolution strategies for playing atari. arXiv preprint [arXiv:1802.08842](https://arxiv.org/abs/1802.08842)
48. Couzin ID, Krause J, James R, Ruxton GD, Franks NR (2002) Collective memory and spatial sorting in animal groups. *J Theor Biol* 218(1):1–11

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.